Analyser — Developer Guide

> **Version:** 2.0 · **Last updated:** June 2026

This guide covers the technical internals of the Analyser application for contributors and maintainers. It focuses on areas not already covered by inline code comments.

Table of Contents

---

1. Project Setup

```
npm install
npm run dev        # dev server on http://localhost:5173
npm run build      # production build — also generates stats.html bundle report
npm run check      # svelte-check + tsc
npm test           # vitest unit tests
```

`npm run build` generates `stats.html` (via `rollup-plugin-visualizer`) in the project root. Open it in a browser after any build to inspect chunk sizes and identify modules that have regressed into the initial bundle. The file is gitignored.

The app uses **SvelteKit** with the **Vercel adapter** (`@sveltejs/adapter-vercel`). Deployment is to Vercel via GitHub Actions on push to `main`.

---

2. Architecture Overview

```
src/
├── lib/
│   ├── fit/          # FIT file parsing (Web Worker, normalisation, device classific
│   ├── align/        # Activity alignment (timestamp sync, distance interpolation)
│   │                 # - distance.ts: interpolateToDistanceAxis, distanceStep(totalD
│   │                 #   distanceStep: ≤50 km → 10 m step; >50 km → 20 m step (halve
│   ├── analytics/    # Smoothing, mean/max curves, summary statistics, anomaly detec
│   │                 # - anomalies.ts: detectAnomalies(), groupAnomalyEvents(), grou
│   │                 # - zones.ts: hrZone, hrZoneFromMaxHR, hrZoneFromLTHR, lthrToEs
│   │                 #   cpZone, ftpZone (7-zone Coggan), ftpPct, cpPct, wPerKg,
│   │                 #   hrZoneBoundaries, cpZoneBoundaries, ftpZoneBoundaries (7 ba
│   │                 #   All pure functions — no side effects. Re-exported from anal
│   │                 # - recordFilter.ts: applyRecordFilter(), deriveGradients(), fi
│   │                 #   Pure functions for applying RecordFilter to ActivityRecord
│   │                 # - rss.ts: computeRSS(durationS, avgPower, cp) — Running Stres
│   │                 #   Formula: (durationS × avgPower × IF²) / (CP × 3600), IF = a
│   │                 #   Returns 0 when any input is zero or negative.
│   │                 # - segmentStats.ts: computeSegmentStats(activity, segment, pro
│   │                 #   Computes time, avgPace, avgPower, maxPower, powerPct, avgHR
│   │                 # - stats.ts: mean(), stdDev() — population statistics ignoring
│   │                 # - downsample.ts: downsampleGps(points, maxPoints, epsilon) —
│   │                 #   simplification; GPS_MAX_POINTS=500 hard cap; iterative RDP
│   │                 #   no recursion limit) with uniform-decimate fallback if RDP s
│   ├── compare/      # Delta computation, segment analysis
│   ├── export/       # Client-side data export
│   │   ├── columns.ts         # Shared column utilities: CHANNEL_KEYS, presentChanne
│   │   │                      #   formatCellValue(), escapeCsvField(); ExportFormat
│   │   ├── csv.ts             # buildCsv(activities) → RFC 4180 string; single-file
│   │   │                      #   multi-file prepends it; CRLF line endings; channel
│   │   ├── excel.ts           # buildWorkbook(activities) → ArrayBuffer (.xlsx via S
│   │   ├── exportActivities.ts # exportActivities(activities, format): lazy-loads cs
```

```
│   │   │                       #   triggers download; format defaults to 'csv'
│   │   └── download.ts         # triggerDownload(), downloadPng(), localDateString()
│   ├── components/
│   │   ├── charts/   # ECharts wrappers (ECharts is lazy-loaded — see §2.1)
│   │   │             # - echarts-loader.ts: singleton loadECharts() — one dynamic im
│   │   │             # - chart-skeleton.css: shared shimmer skeleton styles used by
│   │   │             # - TimeSeriesChart.svelte: per-series stats row (min/avg/max,
│   │   │             #   activeRecordIndices prop (Map<activityId, Set<number>>): wh
│   │   │             #   not in the passing set render as grey 15%-opacity inactive
│   │   │             #   All line series use sampling: 'lttb' (Largest-Triangle-Thre
│   │   │             #   automatic ECharts-side downsampling on large datasets (>200
│   │   │             # - TimeSeriesChart.utils.ts: computeSeriesStats(), formatStat\
│   │   │          #   selectFtpBands(bands, useFullFtpZones): sorts + slices to 5 or
│   │   │          #   computeZoneAxisCap(useFullFtpZones, ftpDataMax, zoneAxisMax): E
│   │   │          #   shouldUseFullFtpZones(data, ftp): true when any point exceeds 1
│   │   │             # - StripChart.svelte: metric strip chart wrapper (map tab)
│   │   │             # - StripToggle.svelte: line/gradient pill toggle
│   │   │             # - StripChart.utils.ts: shouldShowGradient(), GRADIENT_COLOUR_
│   │   │             # - png-btn.css: shared PNG download button styles
│   │   ├── map/      # Leaflet map + ActivityMap.component.test.ts (jsdom)
│   │   │             # - ActivityMap.utils.ts: extractGpsPoints, metricValuesForGpsP
│   │   │             #   smoothedValues) — maps downsampled GPS points to metric val
│   │   │             # - DeviceToggleBar.utils.ts: buildAnomalyCounts(activities) →
│   │   │             #   aggregates event-level anomaly counts across all activities
│   │   └── ui/       # Layout components, controls, SyncPanel
│   ├── server/
│   │   └── redis.ts  # Upstash Redis singleton (server-only)
│   ├── stores/
│   │   ├── filterStore.ts   # Record filter store (see §3h)
│   │   │                     # - recordFilter: writable<RecordFilter>({})
│   │   │                     # - activeFilterCount: derived — counts active channel
│   │   │                     # - clearAllFilters(): resets recordFilter to {}
│   │   ├── athleteProfile.ts # Athlete profile store + localStorage persistence (see
│   │   │                     # - athleteProfile: writable<AthleteProfile>({})
│   │   │                     # - setProfileField(key, value): updates store + persis
│   │   │                     # - initAthleteProfile(): loads from localStorage; SSR-
│   │   │                     # - Storage key: 'analyser-athlete-profile'; quota erro
│   │   ├── customSegments.ts # Custom event segment CRUD + localStorage (see §3g)
│   │   │                     # - Storage key: 'analyser-custom-segments'; keyed by c
│   │   │                     # - getSegments/addSegment/renameSegment/resizeSegment/
│   │   │                     # - getAllSegments/replaceAllSegments/setOnSegmentChang
│   │   ├── deviceLabels.ts  # Device label persistence (localStorage)
│   │   ├── sync.ts          # Cross-device sync logic (labels + segments)
│   │   ├── session.ts       # Activity session state; activityColourMap assigns stab
│   │   ├── toast.ts         # Transient toast notification store (addToast, removeTo
│   │   └── viewport.ts      # Responsive breakpoint store
│   ├── utils/        # Pure utility functions
│   │                 # - binarySearch.ts: lowerBound<T>(arr, key, target) — shared l
```

```
│   │                # - channels.ts: deriveAvailableChannels()
│   │                # - deviceChannels.ts: deviceKey(), deriveDeviceLabel(), groupS
│   │                #   deriveDeviceLabel() is source-aware for power channels: ret
│   │                #   '[Manufacturer] Running Power', or standard label based on
│   │                # - fetchWithRetry.ts: fetchWithRetry(), isRetryable(), compute
│   │                # - formatAge.ts: formatAge(ts) → human-readable relative time
│   │                # - formatting.ts: formatPace(decimalMinutes) → "M:SS" string -
│   │                # - indoorWarnings.svelte.ts: createIndoorWarnings() composable
│   │                # - lapMarkers.ts: buildLapMarkers()
│   │                # - segments.ts: buildSegments(activity, customSegments?) — aut
│   │                #   then custom segments appended with custom: true flag and id
│   │                # - summaryContext.ts: buildCellContext(ch, avg, sport, profile
│   │                #   Derives % FTP / % CP label, w/kg, and HR zone for a summary
│   │                #   Returns null when no relevant profile field applies.
│   ├── validation.ts # Shared regex constants (UUID, short code)
│   └── types.ts      # Domain types
│                     # - Anomaly, AnomalyType, DetectionStrategy, AnomalyDetectionOp
│                     # - ChannelKey now includes 'position' (GPS-only; no numeric se
│                     # - CustomSegment { id: string; name: string; startDist: number
│                     # - GpsPointWithDistance: lat, lon, distance, recordIndex (O(1)
│                     # - GpsPointWithMetric extends GpsPointWithDistance: metricValu
└── routes/
    ├── api/
    │   └── labels/
    │       ├── [uuid]/+server.ts          # GET / PUT label store
    │       └── resolve/[code]/+server.ts  # GET short-code resolver
    ├── export-btn.css   # Shared CSS for the Export Data button in tab bars
    ├── map-panel.css    # Shared CSS for Map tab layout (.map-wrap--has-strip, .stri
    ├── +layout.svelte   # App shell, initSync, ?sync= param
    ├── +page.svelte     # Landing / drop zone
    ├── compare/+page.svelte
    └── event/+page.svelte
```

2.1 ECharts Lazy-Loading

ECharts (≈1.1 MB / 374 kB gzipped) is **not in the initial bundle**. It is deferred to a separate async chunk and fetched only when the first chart component mounts.

How it works

All four chart components (`TimeSeriesChart` , `DeltaChart` , `MeanMaxChart` , `SegmentChart`) import ECharts through a shared singleton loader:

```
// src/lib/components/charts/echarts-loader.ts
import type * as EChartsNamespace from 'echarts';
export type EChartsModule = typeof EChartsNamespace;

let cached: Promise<EChartsModule> | null = null;
```

```
export function loadECharts(): Promise<EChartsModule> {
  if (!cached) {
    cached = import('echarts') as Promise<EChartsModule>;
  }
  return cached;
}
```

Each chart component calls `loadECharts()` inside an `async onMount`:

```
onMount(async () => {
  ec = await loadECharts();               // dynamic import (first call fetches; subs
  chart = ec.init(container, undefined, { renderer: 'canvas' });
  chart.setOption(buildOption(), { notMerge: true }); // explicit initial render
  // ... event binding, ResizeObserver, etc.
  ready = true;
});
```

Why `chart.setOption()` is called explicitly in `onMount`

Three of the four chart components (`TimeSeriesChart`, `DeltaChart`, `SegmentChart`) declare `chart` as a plain `let` variable rather than `$state`. Because `chart` is not reactive, the Svelte `$effect` that normally calls `chart?.setOption()` fires *before* the `await loadECharts()` resolves — hitting a no-op. The explicit `setOption` call immediately after `ec.init()` guarantees the first render regardless of reactive timing.

`MeanMaxChart` uses `let chart = $state<ECharts | undefined>(undefined)` — assigning the chart instance triggers the `$effect` reactively — but still includes the explicit `setOption` call for consistency and to make all four components follow the same pattern.

Loading skeleton

While `loadECharts()` is in flight the chart canvas (`bind:this={container}`) stays in the DOM with `visibility: hidden` so `ec.init()` can be called on an element with real dimensions. A shimmer placeholder div replaces the visual space:

```
{#if !ready}
  <div class="chart-canvas chart-skeleton" aria-hidden="true"></div>
{/if}
<div bind:this={container} class="chart-canvas" style:visibility={ready ? 'visible' :
```

Skeleton styles live in `src/lib/components/charts/chart-skeleton.css` (imported by all four chart components) and use CSS custom properties defined in `src/routes/layout.css`:

```
/* dark theme (default) */
--skeleton-from: #1e293b;
--skeleton-to:   #334155;

/* light theme */
```

```
  --skeleton-from: #e2e8f0;
  --skeleton-to:   #f1f5f9;
```

Bundle verification

`rollup-plugin-visualizer` is configured in `vite.config.ts`. Every `npm run build` writes `stats.html` to the project root (gitignored). Open it to verify ECharts appears only as a dynamic entry (`isDynamicEntry: true` in the Vite manifest) and is absent from all entry-point sync dependency trees.

Adding a new chart component

If you add a fifth chart component:

1. Import from the shared loader: `import { loadECharts, type EChartsModule } from './echarts-loader'`
2. Keep `import type { ECharts, EChartsOption } from 'echarts'` — type-only imports are erased at compile time and do not create a bundle dependency
3. Use `async onMount`, call `loadECharts()`, call `chart.setOption(buildOption(), { notMerge: true })` immediately after `ec.init()`
4. Import `chart-skeleton.css` and add the skeleton/visibility template pattern

---

3. Cross-Device Label Sync

PR #80 added cross-device sync for device labels. Users rename sensor pills (e.g. "Device 1" → "Polar H10") and those names are shared across all their browsers and devices without requiring an account.

3.1 Design Goals

- **Account-free** — identity is a randomly generated UUID stored in `localStorage`, not tied to any login.
- **Cloud-authoritative** — the Upstash Redis store is treated as the source of truth; local state is overwritten on pull.
- **Fire-and-forget pushes** — label changes auto-push in the background; errors surface in the UI but never block local usage.
- **Human-readable codes** — a short 8-character code (e.g. `E6Y-NXEMF`) derived from the UUID lets users type the code on another device instead of scanning a QR.
- **90-day TTL** — entries in Redis expire after 90 days of inactivity with no explicit deletion needed.

3.2 Data Model in Redis

Two key types are stored:

Key pattern	Value	TTL
`labels:{uuid}`	JSON object: `{ [antDeviceKey]: labelString }`	90 days, rolling
`code:{shortCode}`	UUID string	90 days, rolling

Both keys are refreshed together on every PUT, so activity on one extends the other's TTL automatically.

The `antDeviceKey` format is defined by `deviceKey()` in `src/lib/utils/deviceChannels.ts` — it encodes manufacturer, product ID, and device type into a stable string used as the label map key.

3.3 Identity Lifecycle

```
First visit:
  crypto.randomUUID() → uuid
  deriveShortCode(uuid) → shortCode
  localStorage: SYNC_ID_KEY, SYNC_CODE_KEY
  PUT /api/labels/{uuid} → seed remote with current labels + segments

Returning visit:
  Read uuid / shortCode from localStorage
  GET /api/labels/{uuid} → pull remote labels + segments
    → replaceAllLabels(), replaceAllSegments() (if segments present)

Label change (any visit):
  setOnLabelChange hook → pushLabels(uuid, shortCode) [fire and forget]
    (push includes current segments alongside labels)

Segment change (any visit):
  setOnSegmentChange hook → pushLabels(uuid, shortCode) [fire and forget]

Adopt external identity (QR / copy-link / short code):
  adoptSyncIdentity(uuid) → store uuid in localStorage → pullLabels(uuid)
    (pull applies both labels and segments from remote)

Reset identity:
  DELETE /api/labels/{oldUuid} → remove old labels + segments [fire and forget]
  crypto.randomUUID() → new uuid
  localStorage cleared of old identity
  pushLabels(newUuid, newShortCode) → seed remote under new identity
```

3.4 Sync Store (`sync.ts`)

Location: `src/lib/stores/sync.ts`

Exports:

Export	Type	Description
`syncStatus`	`Writable<SyncStatus>`	Reactive store for UI state (uuid, shortCode, lastSynced, error, syncing)
`initSync()`	`async () => (() => void) | undefined`	Idempotent — safe to call multiple times; second call is a no-op. Returns a cleanup function on first call (call in `onDestroy` to deregister hook and allow HMR reinit); returns `undefined` if already initialised. SSR-safe.
`pushLabels(uuid, shortCode)`	`async`	Upload current labels to Redis. Retries up to 3× on transient errors. Sets `syncStatus.syncing` while in progress. Errors update `syncStatus.error`.
`pullLabels(uuid)`	`async`	Download labels from Redis. Retries up to 3× on transient errors. Sets `syncStatus.syncing` while in progress.
`resolveCode(code)`	`async → uuid`	Resolve short code to UUID via API. Retries up to 3× on transient errors. Throws on 404.
`adoptSyncIdentity(uuid)`	`async`	Replace local identity with external UUID and pull.
`resetSyncIdentity()`	`async`	Fire-and-forget `DELETE` on old UUID, then generate fresh UUID and push current labels + segments under it.
`deriveShortCode(uuid)`	`string`	Pure function: BigInt base-36 encode of UUID, 8 chars, `XXX–XXXXX`.
`getSyncStatus()`	`SyncStatus`	Non-reactive snapshot (for use outside Svelte components).

`SyncStatus` shape:

```
type SyncStatus = {
  uuid:       string | null;
  shortCode:  string | null;
  lastSynced: string | null; // ISO timestamp
  error:      string | null;
  syncing:    boolean;       // true while any sync operation is in flight (including
};
```


3.4a Retry / Backoff (`fetchWithRetry`)

Location: `src/lib/utils/fetchWithRetry.ts`

All three sync network calls (`pushLabels` , `pullLabels` , `resolveCode`) use a shared retry utility rather than bare `fetch` :

```
fetchWithRetry(input, init?, opts?): Promise<Response>
```

Default options: `maxRetries: 3` , `baseDelayMs: 1000` , `jitter: 0.25` (±25%).

Retry classification (`isRetryable`):

Response / Error	Retried?
Network error (`TypeError`)	✅ Yes
5xx	✅ Yes
429	✅ Yes
4xx (except 429)	❌ No
2xx / 3xx	❌ No

Backoff schedule (zero jitter, for clarity):

Attempt	Delay
0 → 1	~1 s
1 → 2	~2 s
2 → 3	~4 s

When the server returns a `429` with a `Retry-After` header, that value (in seconds) overrides the exponential delay for that attempt, preventing wasted retries within the rate-limit window.

After all retries are exhausted: the last `Response` is returned for HTTP errors; the last network error is thrown.

Test isolation: `_setSleepFn(fn)` replaces the internal sleep function so that retry tests do not incur real delays (used in `sync.test.ts` via `beforeEach`).

3.4b Sidebar Sync Indicator

Location: `src/lib/components/ui/Sidebar.svelte`

A persistent status indicator is always visible in the sidebar footer, showing the current sync state without requiring the user to expand the SyncPanel. It subscribes to `$syncStatus` via three `$derived.by()` values:

State	Condition	Icon	Colour
`syncing`	`$syncStatus.syncing === true`	Animated two-arc spinner SVG	Blue (`#3b82f6`)
`error`	`$syncStatus.error !== null`	Triangle warning SVG	Amber (`#fbbf24`)
`ok`	`$syncStatus.lastSynced !== null`	Filled circle SVG	Green (`#10b981`)
`muted`	All others (setting up)	Filled circle SVG	`--color-muted`

The element carries `aria-live="polite"` and a dynamic `aria-label` (e.g. `"Synced just now"` , `"Sync error: ..."`) for screen readers.

When `syncing === true` and `error !== null` simultaneously (i.e. auto-retry is in progress after a previous failure), `SyncPanel.svelte` shows `"Retrying…"` in blue instead of the normal amber `"⚠ Sync error — retry"` link.

3.5 DeviceLabels Integration

Location: `src/lib/stores/deviceLabels.ts`

Three additions support sync:

Function	Description
`getAllLabels()`	Returns a snapshot `Record<string, string>` of all stored labels
`replaceAllLabels(map)`	Overwrites localStorage with the provided map and notifies the store
`setOnLabelChange(fn | null)`	Registers a callback invoked after every `setDeviceLabel` / `removeDeviceLabel` . Pass `null` to deregister.

Only one change callback can be registered at a time. `initSync` registers its push callback after the initial push/pull, so subsequent label changes auto-push. The cleanup function returned by `initSync` calls `setOnLabelChange(null)` to deregister on teardown.

3.6 Toast Notification Store

Location: `src/lib/stores/toast.ts`
Component: `src/lib/components/ui/ToastContainer.svelte`

A lightweight, reusable toast system for surfacing transient non-blocking messages to the user. The store is framework-agnostic — any module can call `addToast` without importing Svelte components.

API

Export	Signature	Description
`toasts`	`Readable<Toast[]>`	Reactive list of active toasts. Read-only — prevents external mutation.
`addToast`	`(message: string, level?: ToastLevel, duration?: number) => void`	Adds a toast. Default level: `'warning'`. Default duration: `5000` ms.
`removeToast`	`(id: number) => void`	Removes a toast immediately and clears its auto-dismiss timer.

Types

```
type ToastLevel = 'info' | 'warning' | 'error';

interface Toast {
  id: number;
  message: string;
  level: ToastLevel;
}
```

Behaviour

- Each toast is assigned a unique auto-incrementing `id`.
- `addToast` schedules a `setTimeout` to call `removeToast(id)` after `duration` ms. The timer handle is stored in a `Map<id, timer>` and cleared by `removeToast` to prevent orphaned timers when a toast is dismissed early.
- `removeToast` with an unknown `id` is a safe no-op.

ToastContainer component

`ToastContainer.svelte` subscribes to `$toasts` and renders the most recent `MAX_VISIBLE` (3) toasts as a fixed-position stack (bottom-right on desktop; full-width above the bottom nav bar on phone). It is mounted **once** in `+layout.svelte` so toasts are visible on every page.

Each toast has a level-appropriate left-border accent and dismiss button (`×`). Colours use hardcoded semantic values (amber/blue/red) against `--color-card` / `--color-text` backgrounds, ensuring correct appearance in both light and dark themes.

Current callers

Caller	Level	Message
`athleteProfile.ts` — `_persist` catch	`warning`	`'Athlete profile could not be saved — storage full'` (QuotaExceededError) or `'Athlete profile could not be saved'` (other errors)
`deviceLabels.ts` — `saveLabels` catch	`warning`	`'Device label could not be saved — storage full'` (QuotaExceededError) or `'Device label could not be saved'` (other errors)
`DropZone.svelte` — parse error catch (single-file and multi-file)	`error`	`'<filename>: <parser error message>'` — fires alongside the existing inline error text so the error persists after navigation
`parser.ts` — `normalise()` negative elapsed	`warning`	`'"<filename>": N record(s) with negative elapsed time were removed.'`
`parser.ts` — `normalise()` out-of-order records	`warning`	`'"<filename>": records were out of order and have been sorted automatically.'`

3.7 API Routes

See the full API reference in `docs/api-reference.md`.

`GET /api/labels/[uuid]` — returns `{ labels, segments? }` or 404. The `segments` field is included when custom segments have been stored for that identity.

`PUT /api/labels/[uuid]` — expects `{ labels: Record<string,string>, shortCode: string, segments?: Record<string, CustomSegment[]> }`. Writes `labels:{uuid}`, `code:{shortCode}`, and (when provided) `segments:{uuid}` to Redis with a 90-day TTL.

`DELETE /api/labels/[uuid]` — deletes `labels:{uuid}` and `segments:{uuid}`. Called by `resetSyncIdentity()` as a fire-and-forget cleanup before establishing a new identity. The `code:{shortCode}` key is left to expire via TTL.

`GET /api/labels/resolve/[code]` — resolves short code to UUID. Rate-limited at 10 req/min/IP via Redis `INCR` / `EXPIRE` pipeline (globally enforced across all Vercel instances). Responds 429 with `Retry-After` header on limit exceeded; fails open if Redis is unavailable.

All routes validate inputs against `UUID_REGEX` / `SHORT_CODE_REGEX` from `src/lib/validation.ts` and return structured JSON errors on 400/404/429/500.

3.8 SyncPanel Component

Location: `src/lib/components/ui/SyncPanel.svelte`

A collapsible panel rendered in the Sidebar footer. It is always mounted (not conditionally rendered) so the sync toggle row is always visible.

Key internal state:

State	Type	Purpose
`isOpen`	`boolean`	Controls panel expansion
`codeInput`	`string`	Bound to the short-code text input
`codeError` / `codeSuccess`	`string|null` / `boolean`	Inline validation feedback
`isResolving`	`boolean`	Disables input during async resolve
`copied`	`boolean`	Briefly true after clipboard write, drives button label

Derived values:

- `syncUrl` — `${origin}?sync=${uuid}`, used for both QR generation and copy-link.
- `qrSvg` — generated client-side via `uqr` (`renderSVG(syncUrl)`). No server round-trip.

The component validates code input against `SHORT_CODE_REGEX` before hitting the API, providing immediate feedback on format errors.

3.9 Layout Wiring

`src/routes/+layout.svelte`

Four wiring points in `onMount`:

1. `initTheme()` — applies the persisted theme preference.
2. `initAthleteProfile()` — loads athlete profile from localStorage into the `athleteProfile` store. SSR-safe; a no-op when `typeof localStorage === 'undefined'`.
3. `initSync()` — idempotent; safe if called more than once (e.g. during HMR). Stores the returned cleanup function in `cleanupSync`.
4. URL parameter handling: if the page loads with `?sync={uuid}`, `adoptSyncIdentity(uuid)` is called and the param is stripped from the URL using `history.replaceState` to avoid sharing it inadvertently via browser history.

`onDestroy` calls `cleanupSync?.()` — deregisters the label-change hook and resets the initialised flag, enabling a clean reinit if the layout remounts (HMR).

3.10 Validation

`src/lib/validation.ts`

Shared between API routes (server) and `SyncPanel` (client) to avoid duplication:

```
UUID_REGEX       = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i
SHORT_CODE_REGEX = /^[A-Z0-9]{3}-[A-Z0-9]{5}$/i
```

3.11 Environment Variables

Variable	Required	Description
`UPSTASH_REDIS_REST_URL`	Yes	REST endpoint for your Upstash Redis database
`UPSTASH_REDIS_REST_TOKEN`	Yes	Read-write token for the database

These are read via `$env/dynamic/private` (SvelteKit server-only) in `src/lib/server/redis.ts`. They must be set in:

- **Local development:** `.env.local` (git-ignored, never committed)
- **Vercel:** Project Settings → Environment Variables

The Redis singleton (`getRedis()`) throws an explicit error if credentials are absent, which the API routes catch and surface as HTTP 500. This prevents silent failures.

3.12 Local Development Setup

1. Create an Upstash account and create a Redis database (free tier is sufficient).
2. Copy the REST URL and token from the Upstash console.
3. Create `.env.local` in the project root:

```
UPSTASH_REDIS_REST_URL=https://your-db.upstash.io
UPSTASH_REDIS_REST_TOKEN=your-token-here
```

4. Run `npm run dev` — sync features are fully functional locally.

> **Tip:** If you do not configure Redis, the app still works — sync API calls will return 500 and `syncStatus.error` will be set, but all FIT file analysis features continue to function normally.

3a. Map Tab — Metric Strip Chart

PR #82 added a co-visible metric strip chart to the Map tab on both `/compare` and `/event` pages. When the user selects a metric in the **Colour by** picker, a chart for that channel slides in below the map with full bidirectional hover sync.

Component hierarchy

```
compare/+page.svelte (or event/+page.svelte)
  └─ .map-panel  (flex column)
       ├─ .map-wrap  (flex: 7)
       │     └─ <ActivityMap
       │           onMetricChannelChange={ch => mapMetricChannel = ch}
       │           hoveredDistance={stripHoveredDistance}
       │         />
       └─ .strip-wrap  (flex: 3, shown only when mapMetricChannel !== null)
             └─ <CollapsiblePanel>  (collapses on tablet/phone)
```

```
└── <StripChart
      channel={mapMetricChannel}
      seriesInputs={stripSeriesInputs}
      lapMarkers={...}
      onHoverDistance={handleStripHoverDistance}
      externalHoverDistance={mapStripHoveredDistance}
    />
```

Hover sync

The strip chart uses a **second, independent hover loop** (`stripHoveredDistance` / `mapStripHoveredDistance`), completely separate from the existing Charts-tab ↔ Map loop (`chartHoveredDistance` / `mapHoveredDistance`). This separation is intentional:

- The existing loop is **gated on** `$xAxisMode === 'distance'` — it only fires when the Charts tab is in distance mode.
- The strip loop is **never gated** — the strip chart always operates in distance mode (`forceDistanceAxis={true}`), so the sync is always valid regardless of the global x-axis setting.

Key design points

Concern	Approach
Strip always uses distance axis	`forceDistanceAxis={true}` prop on `TimeSeriesChart` inside `StripChart`
Isolated zoom/pan	`groupId="map-strip"` — separate ECharts connect group from `"compare-charts"` / `"event-charts"`
Gradient mode	`shouldShowGradient()` utility in `StripChart.utils.ts` ; single-file only (multi-file stays as lines)
Shared layout CSS	`.map-wrap--has-strip` and `.strip-wrap` styles extracted to `src/routes/map-panel.css` and imported in both pages
Magic-string safety	Gradient sentinel value is `GRADIENT_COLOUR_TOKEN = '__gradient__'` (exported constant, not inline string)

`ActivityMap` — `onMetricChannelChange` callback

`ActivityMap.svelte` accepts an optional `onMetricChannelChange?: (channel: ChannelKey | null) => void` prop. It fires via a `$effect` whenever the internal `metricChannel` state changes (on picker selection and on file change/reset). Existing callers that do not pass this prop are unaffected.

3b. Data Export

PR #83 added Excel workbook and per-chart PNG downloads, PR #111 added CSV export and extracted shared column utilities into `columns.ts` .

Module layout

File	Responsibility
`src/lib/export/columns.ts`	Shared column utilities used by both CSV and Excel builders: `CHANNEL_KEYS`, `presentChannels()`, `buildHeaderLabel()`, `formatCellValue()`, `escapeCsvField()`; `ExportFormat` type (`'csv' | 'xlsx'`)
`src/lib/export/csv.ts`	`buildCsv(activities)` — builds an RFC 4180 CSV string in memory
`src/lib/export/excel.ts`	`buildWorkbook(activities)` — builds a SheetJS workbook in memory and returns it as an `ArrayBuffer`
`src/lib/export/exportActivities.ts`	`exportActivities(activities, format)` — async entry point: lazy-imports `csv.ts` or `excel.ts` based on `format` (defaults to `'csv'`), then calls `triggerDownload`
`src/lib/export/download.ts`	`triggerDownload(data, filename, mime)`, `downloadPng(dataUrl, filename)`, `localDateString()`

Both `csv.ts` and `excel.ts` are lazy-imported in `exportActivities.ts` so that SheetJS and the CSV builder are only bundled into dynamically loaded chunks, keeping the main bundle lighter.

Shared column utilities (`columns.ts`)

`columns.ts` is the single source of truth for which channels appear in exports and how values are formatted. Both `csv.ts` and `excel.ts` import from it to prevent drift as new channels are added.

- **`CHANNEL_KEYS`** — ordered list of all exportable `ChannelKey` values
- **`presentChannels(records)`** — filters `CHANNEL_KEYS` to those with at least one non-null value across the provided records
- **`buildHeaderLabel(key)`** — returns the human-readable column header (e.g. `"Heart Rate (bpm)"`)
- **`formatCellValue(key, value)`** — formats a raw value for output: pace → `"M:SS"` string; timestamps → `Date`; others → number or `null`
- **`escapeCsvField(value)`** — wraps a string in double-quotes and doubles any internal quotes per RFC 4180; returns the original string if no quoting is needed

CSV structure (`csv.ts`)

`buildCsv(activities)` produces an RFC 4180 string:

- **Single activity** — header row + one row per `ActivityRecord`; no `activity` column
- **Multiple activities** — header row + all records from all activities in order; `activity` column (source filename) prepended as column 1

- Channels with all-null values across all records are omitted
- Pace formatted as `"M:SS"`; timestamps as ISO 8601 strings
- Lines separated by CRLF; file ends with a trailing CRLF

Excel workbook structure (`excel.ts`)

`buildWorkbook` produces:

1. **Summary sheet** — one row per activity with filename, sport, start time (Excel datetime), distance (km), and elapsed time (H:MM:SS or M:SS).
2. **Per-activity sheets** — one row per `ActivityRecord`. Sheet names are taken from `activity.filename`, truncated to 31 characters (Excel limit), and de-duplicated with `(N)` suffixes if needed.

Column selection is dynamic: `presentChannels()` from `columns.ts` filters to channels with at least one non-null value in the activity.

Pace is formatted as an `"M:SS"` string by `formatCellValue()` rather than stored as a decimal, since Excel has no built-in pace format and decimal values are not human-readable.

Timestamps use JavaScript `Date` objects with `cellDates: true` in the SheetJS write options, with `yyyy-mm-dd hh:mm:ss` cell format applied to datetime columns.

PNG download

Each chart component calls `chart.getDataURL({ type: 'png', pixelRatio: 2, backgroundColor: bgColour })` on its ECharts instance, then passes the resulting base64 data URL to `downloadPng()`. `downloadPng` decodes the base64 string to a `Uint8Array`, wraps it in a `Blob`, and calls `triggerDownload` to initiate the browser download.

The background colour is read from the CSS custom property `--color-bg-primary` at download time so that the PNG matches the user's current theme.

`localDateString()`

`new Date().toISOString()` returns UTC, which rolls over to the next day before midnight for users in UTC− timezones. `localDateString()` uses `toLocaleDateString('en-CA')` instead, which produces an ISO-format (`YYYY-MM-DD`) date in the user's local timezone.

3c. Chart Stats Row

PR #85 added a compact stats row below each `TimeSeriesChart` showing minimum, average, and maximum for every active series (added in PR #86). Stats update reactively when devices are toggled, and recalculate when the user zooms into a region.

Pace inversion (PR #88): Because pace is an inverted metric (lower min/km = faster), `computeSeriesStats()` swaps the min/max values for the `pace` channel so that `SeriesStats.max` holds the fastest (numerically smallest) pace and `SeriesStats.min` holds the slowest. The template renders pace as `max / avg / min` so numbers still read ascending left-to-right, consistent with the pace chart's inverted y-axis.

Module layout

File	Responsibility
`src/lib/components/charts/TimeSeriesChart.utils.ts`	`computeSeriesStats()`, `formatStatValue()`, `SeriesStats` interface
`src/lib/components/charts/TimeSeriesChart.svelte`	`zoomRange` state, `seriesStats` derived value, `.chart-stats` HTML/CSS

Key functions

`formatStatValue(value, channel)` — formats a raw number for display. Pace channels use `paceFormat()` (M:SS — re-exported from `$lib/utils/formatting` as the shared `formatPace`); integer channels (heartRate, power, cadence) round to zero decimal places; float channels (speed, temperature) show one decimal.

`computeSeriesStats(data, channel, label, colour, xRange?)` — slices `data` to the optional `xRange` window, passes y-values to `summarise()`, and returns a `SeriesStats` object (or `null` if no data). The `xRange` is provided by the zoom handler.

`SeriesStats` interface:

```
interface SeriesStats {
  label:   string;
  colour:  string;
  min:     string;   // formatted for display (pace: slowest; others: numeric minimum
  avg:     string;   // formatted for display
  max:     string;   // formatted for display (pace: fastest; others: numeric maximum
  minRaw:  number;
  avgRaw:  number;
  maxRaw:  number;
  count:   number;
  xMin:    number;
  xMax:    number;
  unit:    string;
}
```

Zoom integration

`TimeSeriesChart.svelte` listens to ECharts' `dataZoom` event. When a zoom is active, the x-axis extent is read from ECharts' internal model via `chart.getModel().getComponent('xAxis', 0)` and stored in `zoomRange`. The `seriesStats` derived value re-runs whenever `zoomRange` changes, slicing each series' data to the visible window.

`zoomRange` is reset to `undefined` on every chart rebuild (new data, smoothing change, axis mode change), so stats always reflect the full dataset after a rebuild.

Data source vs Summary tab

The stats row and the Summary tab use the same `summarise()` function but different data sources:

Surface	Data source	Notes
Stats row	`buildData()`	Smoothed; distance-axis interpolated when in distance mode
Summary tab	`extractChannel(activity.records, ch)`	Raw unsmoothed records

When smoothing is 1 s and the axis is time, the two surfaces agree closely. In distance mode or with smoothing > 1 s, values may differ. Both surfaces carry a `title` tooltip explaining the difference.

3d. Activity Alignment — GPS Anchor and Indoor Detection

PR #91 replaced raw start-time-difference alignment with GPS-anchor-based alignment. PR #92 extended this to handle indoor activities (TrainerRoad, Zwift, treadmill) where GPS is absent or synthetic.

Indoor activity detection

`classifyIndoor(subSport, records)` in `src/lib/fit/parser.ts` determines `activity.isIndoor` at parse time using two signals:

1. **`sub_sport` field** (primary) — if the FIT session contains a recognised indoor sub_sport value, the activity is classified as indoor regardless of GPS presence. The full set is:

   ```
   const INDOOR_SUB_SPORTS = new Set([
     'indoor_cycling', 'virtual_activity', 'spin', 'stationary_bike',
     'treadmill', 'indoor_rowing', 'indoor_running', 'indoor_walking',
     'virtual_ride', 'virtual_run',
   ]);
   ```

   > **Key insight:** Both Zwift (`virtual_activity`) and TrainerRoad (`indoor_cycling`) embed fake GPS coordinates in every record (Zwift uses virtual-world positions; TrainerRoad uses a Central Park, NYC placeholder). GPS presence cannot be used as the indoor detector — `sub_sport` is authoritative.

2. **GPS-absence fallback** — only fires when `sub_sport` is entirely absent (`undefined`) and all records lack a GPS position. Covers older devices that do not write `sub_sport` .

`activity.subSport` stores the raw sub_sport string from the FIT session (or `undefined`). `activity.isIndoor: boolean` is always populated.

Anchor hierarchy

`findAnchor(activity)` in `src/lib/align/anchor.ts` branches on `activity.isIndoor` :

Outdoor path (unchanged from #91):

Priority	Source	`AnchorSource`
1	FIT timer-start event (within 30 s)	`'timer'`
2	First GPS record with speed > 0	`'gpsMovement'`
3	First GPS record (stationary)	`'gpsFix'`
4	File start	`'fileStart'`

Indoor path (added in #92 — GPS signals are skipped entirely):

Priority	Source	`AnchorSource`
1	FIT timer-start event (within 30 s)	`'timer'`
2	First `workout_step` timestamp (within 30 s)	`'workoutStep'`
3	First record with speed/power/cadence > 0	`'indoorMovement'`
4	File start	`'fileStart'`

The result is an `AlignmentAnchor` stored on `Activity` at parse time:

```
type AnchorSource = 'timer' | 'gpsMovement' | 'gpsFix' | 'fileStart'
                  | 'workoutStep' | 'indoorMovement';

interface AlignmentAnchor {
  recordIndex:    number;
  distanceMetres: number;
  elapsedSeconds: number;
  timestamp:      Date;
  source:         AnchorSource;
}
```

`activity.anchor` is computed once in `parser.ts → normalise()` and available to all downstream code. The new `Activity` fields supporting indoor alignment are:

Field	Type	Description
`subSport`	`string | undefined`	Raw FIT `sub_sport` value
`isIndoor`	`boolean`	Computed by `classifyIndoor()` at parse time
`firstIndoorMovementIndex`	`number | null`	First record with speed/power/cadence > 0
`firstWorkoutStepTime`	`Date | null`	Timestamp of first `workout_step` FIT message

Time-axis alignment

`computeAnchoredOffsets(activities)` in `src/lib/align/timestamp.ts` aligns files relative to the first loaded file's anchor timestamp rather than raw `startTime` values.

The `TimeOffsetControl` component shows a colour-coded anchor badge per file. Labels and tooltips are defined in `TimeOffsetControl.utils.ts` as `ANCHOR_LABELS` and `ANCHOR_TITLES` (both typed as `Record<AnchorSource, string>` for compile-time completeness):

Badge	Colour	Anchor source
Timer	Blue	`'timer'`
GPS move	Blue	`'gpsMovement'`
GPS fix	Amber	`'gpsFix'`
File start	Grey	`'fileStart'`
Workout	Green	`'workoutStep'`
Indoor	Green	`'indoorMovement'`

Distance-axis re-zeroing

`interpolateToDistanceAxis(records, axis, distanceOffset)` shifts the distance axis so that `distanceOffset` maps to 0 km. `activity.anchor.distanceMetres` is passed as `distanceOffset` in both Compare and Event pages.

Indoor/mixed session warnings — `createIndoorWarnings()`

`src/lib/utils/indoorWarnings.svelte.ts` exports a Svelte 5 runes composable that encapsulates the shared indoor warning state used by both Compare and Event pages:

```
const indoor = createIndoorWarnings(() => $activities);
// indoor.hasMixedIndoorOutdoor — any indoor + any outdoor file loaded (requires ≥2 f
// indoor.allIndoor             — all loaded files are indoor (single file or more)
// indoor.mixedWarningDismissed — dismissal state for amber banner
// indoor.indoorInfoDismissed   — dismissal state for blue banner
```

The composable owns the dismissal reset `$effect` (fires on every `$activities` change). Each page adds two separate `$effect` blocks for axis auto-switching:

- **Mixed session:** `if (hasMixedIndoorOutdoor && $xAxisMode === 'distance') xAxisMode.set('time')` — locks to Time because mixed distance axes are incompatible.
- **All-indoor session:** `if (allIndoor && untrack(() => $xAxisMode) === 'distance') xAxisMode.set('time')` — defaults to Time on file load but uses `untrack()` so the user can manually switch back to Distance. `untrack()` prevents `$xAxisMode` from becoming a reactive dependency, limiting the effect to firing only when the file set changes.

CSS for the banners is in `src/routes/indoor-warning.css` (shared by both pages).

Proximity warning

`anchorsAreDistant(activities)` returns `true` when any two GPS-anchored activities have anchor positions more than `GPS_PROXIMITY_THRESHOLD_M` (50 m) apart. Both pages show a dismissible amber banner. Indoor activities whose anchors have no GPS position are excluded from the comparison, so a pure-TR file (no GPS) paired with a Zwift file never triggers a false proximity warning.

3e. Data Anomaly Detection

PR #147 added a data quality layer that detects sensor anomalies at parse time and surfaces them in the UI.

Detection engine — `src/lib/analytics/anomalies.ts`

`detectAnomalies(records, options?)` is called once in `normalise()` and returns a sorted `Anomaly[]` stored on `Activity.anomalies`. Detection is O(n) per channel.

Types and strategies:

```
type AnomalyType       = 'spike' | 'dropout' | 'gps-drift';
type DetectionStrategy = 'threshold-relative' | 'statistical';

interface Anomaly {
  channel:           ChannelKey;
  recordIndex:       number;
  type:              AnomalyType;
  value:             number;          // raw sensor value at that record
  detectionStrategy: DetectionStrategy;
}
```

Spike detection — channels: `heartRate`, `power`, `cadence`

Condition	Threshold	Strategy
HR + `options.athleteProfile.maxHR` set	`maxHR + 10 bpm`	`threshold-relative`
Power + `options.powerSource === 'stryd'` + `cp` set	`cp × 1.5`	`threshold-relative`
Fallback (no profile)	`mean + 4σ`	`statistical`

A run above threshold must last ≤ 3 s to be flagged as a spike (longer sustained high values are real effort, not glitches).

Dropout detection — channels: `heartRate`, `power`, `cadence`

A channel value of `0` or `null` for more than 10 consecutive seconds mid-activity (excluding the first/last 30 s warmup/cooldown) is flagged as `type: 'dropout'`, `detectionStrategy: 'threshold-relative'`.

GPS drift detection — channel: `position`

Consecutive GPS positions are compared using the Haversine formula. If the implied speed between two points exceeds 3× the recorded device speed, the record is flagged as `type: 'gps-drift'`, `detectionStrategy: 'threshold-relative'`. GPS drift anomalies use the `'position'` ChannelKey — a GPS-only channel with no numeric chart series. They surface only through the DeviceToggleBar GPS section and as red circle markers on the map.

Options pattern — `AnomalyDetectionOptions`:

```
interface AnomalyDetectionOptions {
  powerSource?:    'stryd' | 'native';
  athleteProfile?: AthleteProfile;   // from $lib/types — weight, ftp, cp, maxHrCycli
}
```

As of PR #152, `detectAnomalies(records, options)` is called from `parser.ts` with `powerSource` populated from the parsed `Activity`. The detection logic is unchanged — only the call site evolved. The `athleteProfile` option is wired and ready; `maxHrCycling` and `maxHrRunning` are used for sport-aware spike thresholds when provided.

Event grouping — `groupAnomalyEvents()`

`detectAnomalies` returns one `Anomaly` per record. A 40-second dropout produces 40 entries. `groupAnomalyEvents(anomalies)` collapses contiguous runs of the same channel + type (consecutive `recordIndex` values differing by ≤ 1) into a single entry (the first record of each run). This converts per-record counts to per-event counts for badge display.

```
// 40 dropout records → 1 dropout event
groupAnomalyEvents(activity.anomalies.filter(a => a.channel === 'heartRate'))
// → [Anomaly { recordIndex: startOfDropout, type: 'dropout', ... }]
```

Both Compare and Event pages call `groupAnomalyEvents()` before passing anomalies to `TimeSeriesChart` (one `markPoint` per event, not per record).

UI surfaces

DeviceToggleBar badges (`src/lib/components/ui/DeviceToggleBar.svelte`)

`buildAnomalyCounts(activities)` in `DeviceToggleBar.utils.ts` aggregates event-level counts across all activities and returns a `Map<ChannelKey, AnomalyCount>`. The badge `⚠ N` is rendered next to each channel label that has anomalies. Tooltip breaks down counts (e.g. `"2 spikes, 1 dropout"`). A standalone **GPS** section appears at the bottom when `anomalyCounts.get('position')` is non-zero — GPS drift anomalies are surfaced here rather than in any numeric channel group.

ChannelToggleBar badges (`src/lib/components/ui/ChannelToggleBar.svelte`)

The Event page passes `anomalyCounts` to `ChannelToggleBar`. Each channel pill shows an inline `⚠N` badge when that channel has anomaly events. No GPS section (Event page has no ActivityMap drift markers).

TimeSeriesChart markPoints (`src/lib/components/charts/TimeSeriesChart.svelte`)

`TimeSeriesChart` accepts `anomalies?: Anomaly[]` (pre-filtered to the chart's channel and pre-grouped via `groupAnomalyEvents`). A red diamond `markPoint` is rendered at each anomaly's x-axis position on the **first visible series**. Positions use `anomalyXValue()` from `TimeSeriesChart.utils.ts` — which returns `elapsedSeconds` in time mode and kilometres in distance mode. Clicking a markPoint zooms the chart to ±15 s (time mode) or ±50 m (distance mode) around the anomaly.

ActivityMap drift circles (`src/lib/components/map/ActivityMap.svelte`)

`ActivityMap` reads `activity.anomalies` directly (no separate prop). GPS-drift anomalies are filtered by `a.type === 'gps-drift'`, and each anomaly's corresponding `ActivityRecord.position` is rendered as a red `L.circleMarker` (radius 5, `#ef4444`). Non-GPS anomalies are ignored on the map.

`position` ChannelKey

`'position'` was added to the `ChannelKey` union to give GPS drift anomalies a first-class channel assignment. It does NOT appear in `ALL_RECORD_CHANNELS` (the ordered list used by `channelsPresentInRecords`), so it is never returned by `deriveAvailableChannels()` and never rendered as a chart series. Its sole purpose is to carry GPS drift anomaly data through `buildAnomalyCounts` to the DeviceToggleBar GPS section.

3f. Athlete Profile

PR #150 added a local athlete profile system: users enter their training thresholds once and the app uses them to annotate charts and summary tables with zone context.

Type — `AthleteProfile`

Location: `src/lib/types.ts`

```
interface AthleteProfile {
  weight?:        number;  // kg
  ftp?:           number;  // W — cycling Functional Threshold Power
  maxHrCycling?:  number;  // bpm — cycling max heart rate
  cp?:            number;  // W — running Critical Power (Stryd model)
  maxHrRunning?:  number;  // bpm — running max heart rate
  lthr?:          number;  // bpm — Lactate Threshold Heart Rate (fallback)
}
```

All fields are optional. The store starts as `{}` and any field can be removed by setting it to `undefined`.

Store — `athleteProfile.ts`

Location: `src/lib/stores/athleteProfile.ts`

Export	Description
`athleteProfile`	`Writable<AthleteProfile>` — reactive store
`setProfileField(key, value)`	Updates one field in the store and persists to localStorage. Deletes the key when `value` is `undefined` or `NaN`. Emits a `'warning'` toast on `QuotaExceededError`.
`initAthleteProfile()`	Loads persisted profile from localStorage. SSR-safe (`typeof localStorage === 'undefined'` guard). Called once in `+layout.svelte` `onMount`.

Storage key: `'analyser-athlete-profile'`. Stored as a JSON object (only set keys are present — unset fields are omitted).

Zone utility module — `zones.ts`

Location: `src/lib/analytics/zones.ts` (re-exported via `src/lib/analytics/index.ts`)

All functions are pure with no side effects. Every denominator function returns `0` for zero or negative input to prevent `Infinity` results.

Function	Signature	Description
`hrZone`	`(bpm, profile, sport) → HrZone | null`	Sport-aware HR zone. Uses `maxHrCycling` for `'cycling'`, `maxHrRunning` otherwise. Falls back to the other sport's maxHR, then to LTHR. Returns `null` when no threshold is set.
`hrZoneFromMaxHR`	`(bpm, maxHR) → HrZone`	5-zone % of maxHR model (Z1 < 60%, …, Z5 ≥ 90%)
`hrZoneFromLTHR`	`(bpm, lthr) → HrZone`	5-zone % of LTHR model (Z1 < 81%, …, Z5 ≥ 100%)
`lthrToEstimatedMaxHR`	`(lthr) → number`	`lthr / 0.92` — Friel/Coggan approximation. Use when maxHR is unavailable.
`cpZone`	`(watts, cp) → CpZone`	Stryd 5-zone CP model (Z1 < 75%, …, Z5 ≥ 120%)
`ftpPct`	`(watts, ftp) → number`	`round((watts / ftp) * 100)`. Returns `0` when `ftp ≤ 0`.
`cpPct`	`(watts, cp) → number`	`round((watts / cp) * 100)`. Returns `0` when `cp ≤ 0`.
`wPerKg`	`(watts, weightKg) → number`	`round((watts / weightKg) * 10) / 10`. Returns `0` when `weightKg ≤ 0`.
`ftpZone`	`(watts, ftp) → FtpZone`	Coggan 7-zone FTP model (Z1 < 55%, Z2 55–75%, Z3 75–90%, Z4 90–105%, Z5 105–120%, Z6 120–150%, Z7 ≥ 150%)
`hrZoneBoundaries`	`(maxHR) → ZoneBand[]`	5 contiguous `{ min, max, zone }` bands for ECharts `markArea` construction
`cpZoneBoundaries`	`(cp) → ZoneBand[]`	5 contiguous `{ min, max, zone }` bands for ECharts `markArea` construction (Stryd model)
`ftpZoneBoundaries`	`(ftp) → ZoneBand[]`	7 contiguous `{ min, max, zone }` bands for ECharts `markArea` construction (Coggan 7-zone model)

How the profile flows into the UI

All three consumers receive `athleteProfile` and `sport` as props from their respective page components:

```
<!-- compare/+page.svelte -->
<TimeSeriesChart athleteProfile={$athleteProfile} sport={$activities[0]?.sport ?? ''}
<MeanMaxChart    athleteProfile={$athleteProfile} sport={$activities[0]?.sport ?? ''}
```

`TimeSeriesChart.svelte` — adds zone `markArea` bands per-series using each series' own `s.activity.sport` (not a chart-level `sport` prop). Running activities get CP zone bands (`cpZoneBoundaries`); cycling activities get adaptive FTP zone bands (`ftpZoneBoundaries` — see

below); HR channels get maxHR bands (`hrZoneBoundaries`), with cross-sport fallback and LTHR path. Each series attaches its own `markArea` so mixed-sport comparisons shade correctly.

Adaptive FTP zone shading (PR #159): `buildOption()` pre-computes all series data into a `dataCache` Map before zone/axis decisions. For cycling power charts with an FTP set, it scans the cache to find `ftpDataMax` (the max power value across all cycling series). Then:

- `useFullFtpZones = ftpDataMax > ftp * 1.20` — when true, all 7 Coggan zones are shown and `zoneAxisMax` is cleared so the y-axis auto-scales to data.
- When false, only Z1–Z5 are rendered and `zoneAxisMax` clips the y-axis at ~120% FTP for a compact view.

Two pure helpers in `TimeSeriesChart.utils.ts` support this:

Helper	Signature	Description
`selectFtpBands(bands, useFullFtpZones)`	`<T extends { zone: number }>(T[], boolean) → T[]`	Sorts bands by zone number and returns all 7 (7-zone mode) or the first 5 (5-zone mode). Does not mutate the input.
`computeZoneAxisCap(useFullFtpZones, ftpDataMax, zoneAxisMax)`	`(boolean, number, number | undefined) → number`	Returns `Math.ceil(ftpDataMax × 1.2)` in 7-zone mode (bounds the open-ended Z7 `markArea` band without ECharts `Infinity` errors), or `zoneAxisMax ?? 9999` in 5-zone mode.

The `dataCache` Map (`Map<number, [number, number | null][]>`) also eliminates redundant `buildData()` calls — the series loop reads from cache instead of recomputing. HR (5-zone) and CP running power (5-zone) zones are completely unaffected by this logic.

`MeanMaxChart.svelte` — adds an FTP (cycling) or Critical Power (running) dashed `markLine`. Adds a second w/kg Y-axis when `weight` is set and `weight > 0` .

`src/lib/utils/summaryContext.ts` — `buildCellContext(ch, avg, sport, profile)` is the shared utility used by both compare and event pages to derive the Summary tab context object:

```
// For power channels:
{ pctLabel: '92% FTP' | '88% CP', wkg: '3.6' }

// For heartRate:
{ zone: 2 }

// Returns null when no relevant profile field is set
```

`src/lib/analytics/rss.ts` — `computeRSS(durationS, avgPower, cp)` computes the Running Stress Score shown in the Summary tab for running activities when CP is set. Returns 0 for zero or negative inputs.

Sidebar SSR workaround

`AthleteProfilePanel` is wrapped in `{#if mounted}` in `Sidebar.svelte`:

```
let mounted = $state(false);
onMount(() => { mounted = true; });

{#if mounted}<AthleteProfilePanel />{/if}
```

This prevents an SSR hydration crash where the panel's `<button>` element was absent from server-rendered HTML, causing Svelte 5's `sibling()` to consume the wrong DOM node. The workaround causes the Profile toggle button to appear only after client hydration (brief FOUC on first load). Tracked in issue #151.

3g. Custom Event Segments

PR #156 added user-defined course sections ("custom segments") to the Event Comparison view. Users hold Shift and drag on any time-series chart to select a distance range, name the segment, and have it persist across sessions for the same course.

Domain type — `CustomSegment`

Location: `src/lib/types.ts`

```
interface CustomSegment {
  id:        string;  // UUID generated at creation
  name:      string;  // user-supplied label
  startDist: number;  // metres from activity start
  endDist:   number;  // metres from activity start
}
```

Segment store — `customSegments.ts`

Location: `src/lib/stores/customSegments.ts`

Segments are stored in `localStorage` under the key `'analyser-custom-segments'` as a `Record<courseKey, CustomSegment[]>`. An in-memory write-through cache (`_cache`) avoids repeated JSON parse on every read.

Course identity key

```
courseKey(activity: Activity): string
// Format: `${sport}:${Math.round(totalDistance / 100)}`
// Example: 'running:50' for a 5.0 km parkrun
```

Rounding to the nearest 100 m tolerates GPS drift between sessions: a course recorded as 5.01 km and one as 4.98 km both resolve to `'running:50'`, sharing the same segment set.

Public API

Export	Description
`courseKey(activity)`	Derive the stable course identity key
`getSegments(key)`	Return a shallow copy of all segments for a course (empty array if none)
`addSegment(key, name, startDist, endDist)`	Add a new segment; generates a UUID id. Fires `_onSegmentChange`
`renameSegment(key, id, newName)`	Rename by id; no-op if id not found. Fires `_onSegmentChange`
`resizeSegment(key, id, startDist, endDist)`	Update distance bounds by id; no-op if id not found. Fires `_onSegmentChange`
`removeSegment(key, id)`	Remove by id; no-op if id not found. Fires `_onSegmentChange`
`getAllSegments()`	Return a shallow copy of the full cache (for sync serialisation)
`replaceAllSegments(map)`	Bulk-overwrite cache and localStorage (sync pull — does NOT fire onChange)
`setOnSegmentChange(fn | null)`	Register/deregister the callback invoked after every user-initiated write

`getSegments()` returns a copy (not the live array) so Svelte reactivity can detect changes when callers reassign their local variable from the return value.

`localStorage` quota errors surface as `'warning'` toasts via `addToast()`, matching the pattern used by `deviceLabels.ts` and `athleteProfile.ts`.

Sync integration

`customSegments.ts` mirrors the `deviceLabels.ts` pattern: `setOnSegmentChange` lets `sync.ts` register a push callback during `initSync()`. After that, any user-initiated segment write (add/rename/resize/remove) automatically triggers a `pushLabels()` call that includes the full segment map alongside labels. On pull, `replaceAllSegments()` is called when the response contains a `segments` field.

Segment stats engine — `segmentStats.ts`

Location: `src/lib/analytics/segmentStats.ts`

`computeSegmentStats(activity, segment, profile?)` returns a `SegmentStats` object for one activity over one segment:

```
interface SegmentStats {
  time:        number;         // seconds
  avgPace:     number | null;  // min/km
  avgPower:    number | null;  // watts
  maxPower:    number | null;  // watts
  powerPct:    number | null;  // % CP (running) or % FTP (cycling); null when no pro
```

```
  powerLabel:  string;         // 'Stryd' | 'Running Power' | 'Power'
  avgHR:       number | null;  // bpm
  hrZones:     number[] | null; // [z1%, z2%, z3%, z4%, z5%] — null when no HR thresh
}
```

Time computation uses `timeAtDistance()` from `src/lib/compare/delta.ts` — linear interpolation on the distance axis, matching the same logic used by the delta chart. This ensures the segment time matches the delta plot value.

Power label is derived from `activity.powerSource` (`'stryd'` → `'Stryd'`; `'native'` → `'Running Power'`; otherwise `'Power'`).

HR zone distribution is computed only when `profile` is provided and at least one of `maxHrRunning`, `maxHrCycling`, or `lthr` is set. Each record in the segment is classified into a zone via `hrZone()` from `src/lib/analytics/zones.ts`, and the result is expressed as integer percentages summing to ≈100.

`buildSegments()` — custom segment merging

Location: `src/lib/utils/segments.ts`

`buildSegments(activity, customSegments?)` accepts an optional `CustomSegment[]` second argument. Custom segments are appended **after** auto-lap or kilometre-split segments, each tagged with `custom: true` and `id`:

```
// Auto-lap result:
{ label: 'Lap 1', startDist: 0, endDist: 1000 }

// Custom segment result:
{ label: 'The Hill', startDist: 1200, endDist: 2400, custom: true, id: 'abc-123' }
```

The `custom` and `id` fields are used by:

- `event/+page.svelte` to identify which segments to show resize handles for
- `SegmentManager.svelte` to identify which segments can be renamed/deleted

`SegmentManager.svelte` component

Location: `src/lib/components/ui/SegmentManager.svelte`

A collapsible toolbar panel that lets users view, rename, delete, export, and import custom segments for the current course. It is shown only on the Event page when `currentCourseKey` is non-null (i.e. at least one activity is loaded).

Key props:

Prop	Type	Description
`courseKey`	`string`	The active course identity key
`segments`	`CustomSegment[]`	Current segment list (read-only display)
`onchange`	`() => void`	Called after any mutation so the page re-reads from the store

Export writes a JSON file (`analyser-segments-<date>.json`) containing a `{ courseKey, segments }` envelope. Import validates that `startDist < endDist` for every entry and rejects the file with a toast error if any segment is malformed.

`TimeSeriesChart.svelte` — Shift+drag and resize

Location: `src/lib/components/charts/TimeSeriesChart.svelte`

Two new interaction modes are added when the `resizeActive` prop is `true` (Event page only):

Shift+drag — segment creation

When the user holds Shift, ECharts brush mode is enabled (`brushType: 'lineX'`). On brush end, the selected x-axis range is converted to metres and emitted via the `onBrushSelect` prop. The Event page receives this and opens a name-prompt dialog.

ECharts `connect()` links all chart instances in the `groupId` group. To prevent a brush selection on one chart from firing `brushEnd` on all connected charts, a module-level `_brushGroupHandledMs` map (keyed by `groupId`) deduplicates the event: only the first handler within the same millisecond advances; subsequent calls are no-ops.

Resize drag handles

When `customSegmentBands` are passed to the chart, each band edge becomes a drag target:

1. A ZRender `mousemove` handler checks proximity (< 8 px) to any segment boundary and sets `cursor: col-resize` .
2. A ZRender `mousedown` handler (when near a boundary) begins a drag: stores `{ segId, side, oppositeKm }` , disables `dataZoom inside` , and calls `e.stop()` to prevent ECharts from also handling the event.
3. Window-level `mousemove` / `mouseup` handlers track the drag out-of-canvas and commit the resize via the `onSegmentResize` prop on `mouseup` .
4. `onDestroy` removes both window listeners and cleans up the `_brushGroupHandledMs` entry for the chart's group.

`onSegmentResize(id, newStartDist, newEndDist)` is called with distances in metres; the Event page handler calls `resizeSegment()` and re-reads the store.

3h. Record Filtering

PR #157 added a real-time record filter that lets users isolate segments of interest — threshold power efforts, climbing sections, high-cadence intervals — by constraining one or more channels. The filter applies to both the Compare and Event pages.

Filter store — `filterStore.ts`

Location: `src/lib/stores/filterStore.ts`

```
interface ChannelRange {
  min?: number;
  max?: number;
```

```
}

interface RecordFilter {
  speed?:     ChannelRange;  // km/h (or min/km for running pace)
  power?:     ChannelRange;  // W
  heartRate?: ChannelRange;  // bpm
  cadence?:   ChannelRange;  // spm (running) or rpm (cycling)
  gradient?:  ChannelRange;  // % grade derived from altitude+distance
  inverted?:  boolean;       // when true, records that FAIL all rules are shown inst
}
```

Export	Type	Description
`recordFilter`	`Writable<RecordFilter>`	Global filter state; `{}` means no filter active
`activeFilterCount`	`Derived<number>`	Count of channels with at least one bound set (0–5); drives the badge on the toggle button
`clearAllFilters()`	`() => void`	Resets `recordFilter` to `{}`

Filter engine — `recordFilter.ts`

Location: `src/lib/analytics/recordFilter.ts`

Three pure functions, zero side effects:

Function	Signature	Description
`applyRecordFilter`	`(records, filter, gradients?) → Set<number>`	Returns the set of record indices that pass all active constraints. Missing values (`null` / `undefined`) pass by default. With `inverted: true` , the logic is flipped — the returned set contains indices that would normally fail.
`deriveGradients`	`(records) → (number | null)[]`	Derives % grade per record from consecutive altitude+distance deltas. Returns `null` where altitude is absent or distance did not change. O(n).
`filterRecords`	`(records, passingIndices) → ActivityRecord[]`	Returns only the records whose index is in the passing set. Used where a filtered array is needed directly (e.g. MeanMax computation).

Gradient passthrough for missing altitude: If a record has no altitude, its gradient entry is `null` and it passes the gradient filter by default. This matches the behaviour for other channels with missing values.

`TimeSeriesChart` — split active/inactive rendering

`TimeSeriesChart.svelte` accepts an optional `activeRecordIndices` prop:

```
activeRecordIndices?: Map<string, Set<number>>
// key: activity.id — value: set of record indices that pass the filter
```

When provided, the chart splits each activity's data into two overlaid series:

Series	Records	Rendering
Active	Indices in `Set<number>`	Full colour, `z: 2`
Inactive	All other indices	15% opacity, grey (`#6b7280`), `z: -1`

The split uses the same `buildData()` function — active series fills in the record values for its indices and `null` elsewhere; inactive series does the inverse. ECharts connects segments across `null` gaps via `connectNulls: false` (default), which causes each contiguous run of active/inactive records to render as a separate segment with a gap at each transition.

`DeltaChart` and `SegmentChart` on the Event page do NOT receive `activeRecordIndices` — the time-delta computation always uses full-course data to preserve meaningful split comparisons.

Reactivity note: Reads inside closures passed to `flatMap` within `buildOption()` are not automatically tracked as Svelte 5 `$effect` dependencies. The chart rebuild effect explicitly declares `void activeRecordIndices;` in its dependency list to force re-evaluation when the filter changes.

`FilterPanel.svelte` component

Location: `src/lib/components/ui/FilterPanel.svelte`

A self-contained collapsible filter control rendered in the toolbar of both Compare and Event pages. It does not use `CollapsiblePanel` — it owns its own `expanded` state and renders its body as a `position: absolute` overlay so it cannot displace charts.

Key props:

Prop	Type	Description
`sport`	`string`	`'running'` or `'cycling'` — controls label text (`Pace` vs `Speed`) and cadence unit (`spm` vs `rpm`)
`powerSource`	`'stryd' | 'native' | 'cycling' | undefined`	Drives the power label (`Stryd Power` , `Running Power` , `Power`)
`athleteProfile`	`AthleteProfile`	Used to build zone preset buttons and named presets
`hasAltitude`	`boolean`	When `false` , the gradient row is hidden (indoor activities have no meaningful gradient)

Named presets — shown when the relevant profile field is set:

Preset	Condition	Filter applied
Above CP	Running + CP set	Power ≥ CP value
Z4+ FTP	Cycling + FTP set	Power ≥ 90% FTP
Z4+ HR	maxHR or LTHR set	HR ≥ 80% maxHR (or ≥ 93% LTHR)
Climbing	hasAltitude	Gradient ≥ 2%
Downhill	hasAltitude	Gradient ≤ −2%

Zone preset buttons — rendered as `Z1`–`Z5` (HR, CP) or `Z1`–`Z7` (FTP) pill buttons below the relevant input row when the corresponding profile threshold is set. Clicking a zone button fills the min/max inputs with the exact zone boundaries and commits immediately.

Filter application uses `oninput` events (not `onchange`) so charts update in real time as the user types. A `clickOutside` Svelte action closes the panel when focus moves elsewhere.

---

4. Responsive Layout

The app is fully responsive across three viewport tiers. Breakpoints are defined in two places that must stay in sync:

- **CSS:** `--bp-phone: 480px` and `--bp-tablet: 768px` in `src/routes/layout.css`
- **JS:** `PHONE_MAX = 480` and `TABLET_MAX = 768` in `src/lib/stores/viewport.ts`

The `viewport` readable store emits `'phone' | 'tablet' | 'desktop'` via `window.matchMedia` listeners. It is SSR-safe (defaults to `'desktop'`).

Key responsive components:

Component	Behaviour change at ≤768px
`Sidebar.svelte`	Switches to `position: fixed` slide-out drawer
`+layout.svelte`	Renders hamburger button (fixed, top-left, 44×44px) and backdrop
`CollapsiblePanel.svelte`	Wraps toolbar controls; `display: contents` on desktop, collapsible on mobile
`+layout.svelte` (phone only)	Renders bottom navigation bar at ≤480px

Sidebar scroll layout: Inside `.sidebar`, the logo and mode-nav buttons are pinned at the top (outside any scroll container). A `.sidebar-scroll` wrapper (`flex: 1; overflow-y: auto; min-height: 0`) wraps the file-section and footer, making the sidebar scrollable when expanded panels (Athlete Profile, Sync) push content below the viewport edge. The file-section retains its own independent `overflow-y: auto` for long file lists. A thin 6px scrollbar is applied via `scrollbar-width: thin` (Firefox) and `::-webkit-scrollbar` (Chrome/Safari/Edge) using `--color-border`.

5. FIT Parsing

FIT files are parsed by `fit-file-parser` running in a **Web Worker** to avoid blocking the UI thread. The parsing pipeline is split across four modules in `src/lib/fit/`:

Module	Role
`parser.ts`	`normalise(data, filename, labels, onStage?)` — pure Worker-safe function; accepts a labels snapshot and an optional stage callback; returns `{ activity: Activity, toasts: ToastMessage[] }`
`parser.worker.ts`	Web Worker script; receives a `ParseWorkerInput`, runs `fit-file-parser` + `normalise()`, posts typed `ParseWorkerMessage` events back to the main thread
`parseInWorker.ts`	Main-thread wrapper; creates one Worker per file, wires up message handlers, returns a `ParseJob` with a cancellable promise
`parseFitFile.ts`	Legacy main-thread bridge; calls `normalise()` synchronously and dispatches toasts directly — preserved for backward compatibility

`index.ts` is a pure barrel that re-exports all four public surfaces.

Worker message protocol — `ParseWorkerMessage` is a discriminated union:

```
type ParseWorkerMessage =
  | { type: 'progress'; stage: ParseStage }
  | { type: 'complete'; activity: Activity; toasts: ToastMessage[] }
  | { type: 'error'; message: string; toasts: ToastMessage[] };

type ParseStage = 'queued' | 'parsing' | 'normalising' | 'detecting_anomalies' | 'bui
```

`parseInWorker(buffer, filename, labels, onStage?)` — creates a Worker, transfers the `ArrayBuffer` (zero-copy), and returns:

```
interface ParseJob {
  promise: Promise<{ activity: Activity; toasts: ToastMessage[] }>;
  cancel: () => void;  // terminates the Worker; rejects promise with ParseCancelledE
}
```

One Worker is spawned per file and terminated immediately on completion, error, or cancel — no pool management. `ParseCancelledError` is a distinct error class so `DropZone.svelte` can catch cancels silently without showing an error toast.

`normalise()` refactor (PR #115) — `normalise()` no longer imports from store modules, making it Worker-safe:

- Accepts `labels: Record<string, string>` (a snapshot of device labels passed at dispatch time) instead of calling `getAllLabels()` internally.
- Accepts an optional `onStage?: (stage: ParseStage) => void` callback for progress reporting.

- Returns `{ activity, toasts }` instead of calling `addToast()` directly — toasts are collected and dispatched by the main-thread caller after the Worker completes.

`deviceKey()` utility (`src/lib/utils/deviceKey.ts`) — shared pure function used by both `parser.ts` and `deviceLabels.ts` to derive a stable label-store key for a device. Priority: `antDeviceNumber` → `serialNumber` → `manufacturer:product` → `antDeviceType` → `null`. Re-exported from `deviceLabels.ts` as `deviceStorageKey` for backward compatibility with existing callers.

```
sequenceDiagram
    participant DZ as DropZone.svelte
    participant PIW as parseInWorker.ts
    participant W as parser.worker.ts
    participant P as parser.ts

    DZ->>DZ: pendingFiles.set(key, { stage: 'queued' })
    DZ->>PIW: parseInWorker(buffer, filename, labels, onStage)
    PIW->>W: new Worker() + postMessage({ buffer, filename, labels })
    W->>W: fit-file-parser.parse(buffer)
    W-->>PIW: { type: 'progress', stage: 'parsing' }
    PIW-->>DZ: onStage('parsing')
    W->>P: normalise(data, filename, labels, onStage)
    P-->>W: { activity, toasts }
    W-->>PIW: { type: 'complete', activity, toasts }
    PIW->>W: worker.terminate()
    PIW-->>DZ: promise resolves → addActivity() + dispatch toasts
    DZ->>DZ: pendingFiles.delete(key)
```

Key parsing details:

- `STRING_DEVICE_TYPE` mapping converts `fit-file-parser`'s snake_case string outputs (e.g. `'heart_rate'`) back to numeric ANT+ constants.
- Device deduplication: duplicate `device_index` entries are discarded (only first occurrence retained).
- Two-pass channel allocation: external sensors (known ANT+ types) claim channels first; the watch claims remaining channels; any still-unclaimed channels are merged into the first device as a safety net.
- Enhanced fields: `enhanced_speed` and `enhanced_altitude` are preferred over `speed` and `altitude`.
- Stryd developer fields: `Power` (capital P), `Form Power`, `stance_time`, `step_length` are mapped to standard channel keys (`power`, `formPower`, `groundContactTime`, `strideLength`).
- **Power source detection:** After record normalisation, `Activity.powerSource` is set: `'stryd'` when any record contained the Stryd `Power` developer field; `'native'` when native `power` exists on a running activity; `'cycling'` when native power exists on a non-running activity; `undefined` when no power data is present. Both Stryd and native power are stored in the single `power` channel — `powerSource` carries the attribution.

Sport-specific post-processing is applied after record normalisation:

Sport	Function	Reason
`running`	`applyRunningCadenceDoubling(records)`	FIT cadence is single-leg (one foot per minute). Doubles to get conventional spm (steps per minute).
`cycling`	`removeCyclingPace(records)`	Pace (min/km) is not a meaningful cycling metric. Clears all `pace` values so the channel is excluded from device streams and charts entirely.

Both functions are exported from `parser.ts` for unit testing.

Record validation is applied in `normalise()` after record normalisation and before sport-specific post-processing (PR #142):

Function	Behaviour	Toast / log
`filterNegativeElapsed(records)`	Returns a new array with all records where `elapsedSeconds < 0` removed. Broken device clocks can emit these.	`console.warn` + `'warning'` toast if any are removed.
`ensureSortedByElapsed(records)`	Sorts `records` in-place by `elapsedSeconds` if out of order; returns `true` when sorting was required.	Caller (`normalise()`) emits `console.warn` + `'warning'` toast on `true` .

Both functions are exported from `parser.ts` for unit testing. Well-formed files pass through both with no output.

Lap building is handled by `buildLaps(fitLaps, records)` , which is also exported for direct unit testing. It derives `startDistance` and `endDistance` from the running cursor position in the records array rather than from the FIT lap's `start_distance` field. This makes it resilient to Garmin devices that write `start_distance = 0` for every lap (per-lap relative) rather than cumulative from the session start. Laps with `total_distance = 0` are returned as zero-length entries (`startIndex === endIndex` , `startDistance === endDistance`) without advancing the cursor.

The cursor advancement uses `records[cursor].distance <= targetDist + DISTANCE_EPSILON_M` (where `DISTANCE_EPSILON_M = 0.5`) to absorb GPS floating-point drift at lap boundaries. The constant is exported for use in tests and any future callers that need the same tolerance.

Anomaly detection is called at the end of `normalise()` after sport-specific post-processing and before `buildLaps()` . See section 3e for details.

Available channel pre-computation — `channelsPresentInRecords(records)` scans each `ActivityRecord` once across all 14 channel keys and returns a `Set<ChannelKey>` of those with at least one non-null value. It is called once inside `normalise()` and the result is stored on `Activity` as `availableChannels: Set<ChannelKey>` (PR #108).

`deriveAvailableChannels(activities)` in `src/lib/utils/channels.ts` unions those pre-computed sets rather than re-scanning records on every render call. It returns the channel list in the canonical ordering defined by `ALL_RECORD_CHANNELS` (exported from `parser.ts`), which also replaces the formerly duplicated `ALL_CHANNELS` constant that used to live only in `channels.ts`.

---

6. Testing

Unit tests use **Vitest**. Test files live alongside the modules they test (e.g. `TimeSeriesChart.test.ts` next to `TimeSeriesChart.svelte`).

```
npm test           # run all tests
npm run test:watch # watch mode
```

Test environments

Most tests run in the default `node` environment (pure utility functions with no DOM). Component tests that mount Svelte components require a DOM and use `jsdom`.

To mark a test file as a component test, add this directive as the **first line** of the file:

```
// @vitest-environment jsdom
```

Why the `browser` resolve condition is needed for component tests:

Svelte 5 exports both a browser build (`index.js`) and a server build (`index-server.js`). In `node` environment, Vite resolves to the server build, which throws `"mount(...) is not available on the server"`. The `vite.config.ts` sets `resolve.conditions: ['browser']` **only when** `mode === 'test'` so that jsdom component tests use the browser build. The production build is unaffected (Vite's built-in defaults include `browser` for client bundles).

```
// vite.config.ts — mode-conditional resolve
export default defineConfig(({ mode }) => ({
  plugins: [tailwindcss(), sveltekit(), visualizer({ filename: 'stats.html', open: fa
  build: { chunkSizeWarningLimit: 1200 }, // ECharts async chunk is ~1.1 MB — suppres
  ...(mode === 'test' ? { resolve: { conditions: ['browser'] } } : {}),
  // ...
}));
```

Mocking Leaflet in component tests:

`ActivityMap.svelte` dynamically imports Leaflet in `onMount`. In jsdom, Leaflet's browser DOM APIs are unavailable. Mock Leaflet at the module level:

```
vi.mock('leaflet', () => { /* minimal stubs for map, tileLayer, control, etc. */ });
vi.mock('leaflet/dist/leaflet.css', () => ({}));
```


Also stub `ResizeObserver` (not implemented in jsdom):

```
vi.stubGlobal('ResizeObserver', class { observe = vi.fn(); disconnect = vi.fn(); });
```

There are no integration or end-to-end tests at present. The API routes for sync are exercised manually — see section 3.11 for local setup.

Shared test helper — `makeBaseActivity`

`src/lib/test-utils.ts` exports `makeBaseActivity(overrides?)`, a single factory for building `Activity` fixture objects in tests (PR #108):

```
import { makeBaseActivity } from '$lib/test-utils';

const activity = makeBaseActivity({ id: 'run-1', records: [/* ... */] });
```

The factory fills every required `Activity` field with safe defaults and computes `availableChannels` automatically via `channelsPresentInRecords(records)`. It also sets `anomalies: []` as the default empty anomaly list, matching `normalise()`'s initial state before `detectAnomalies()` is called. Pass `overrides.availableChannels` to supply a manual set when the test is specifically about channel visibility rather than parser behaviour.

All test files should import this helper rather than duplicating the full `Activity` literal — it prevents future interface changes from requiring edits across every test file.